

Problem Statement:2

Automated DevSecOps Pipeline & Infrastructure Security Compliance for
Cloud-Native Workloads

- A fintech startup is building an API-driven payment processing platform on AWS. Before going live, the platform must comply with **PCI-DSS v4.0** and **SOC 2 Type II** requirements. The DevOps team frequently deploys infrastructure using Terraform, but security reviews are lagging, leading to vulnerable resource deployments (e.g., public S3 buckets, exposed security groups, and unencrypted databases).
- As the Cloud Security & DevSecOps Engineering team, you must design and build an **Automated Security Pipeline and Runtime Runtime Threat Detection Framework**. Your solution will validate Infrastructure as Code (IaC) prior to deployment, enforce secret governance, establish real-time vulnerability management, and automate network security enforcement across microservices.

- **Phase 1: Shift-Left IaC Security & Secret Management (60 Mins)**
- **Pre-Deployment Static Analysis:** Build an **AWS CodePipeline** stage using **AWS CodeBuild** that automatically scans dynamic Terraform or CloudFormation scripts using open-source static analysis tools (e.g., Checkov or tfsec).
- **Secret Governance with AWS Secrets Manager:**
 - Configure pipeline rules to fail the build if hardcoded API keys, passwords, or AWS tokens are detected in code commits using detect-secrets or Trivy.
 - Enforce secret retrieval via **AWS Secrets Manager** with automatic 30-day secret rotation configured via AWS Lambda.
- **Phase 2: Container Security & Vulnerability Scanning (60 Mins)**
- **ECR Image Vulnerability Scanning:** Push container images to **Amazon Elastic Container Registry (ECR)** and enable enhanced scanning powered by **Amazon Inspector**.
- **Automated Deployment Guardrails:**
 - Write an **AWS Lambda** function or build gate that evaluates Amazon Inspector scan findings.
 - Automatically block pipeline deployment to Amazon EKS or ECS if any **CRITICAL** or **HIGH** Common Vulnerabilities and Exposures (CVEs) are detected in the container dependencies or OS layers.

- **Phase 3: Network Security Boundaries & SSM Compliance (60 Mins)**
- **Perimeter Defense with AWS Network Firewall:** Configure an **AWS Network Firewall** endpoints policy in a dedicated VPC inspection subnet to enforce stateful domain filtering and IP dynamic drop rules.
- **Agentless & Agent-Based Patch Audit:** Deploy **AWS Systems Manager (SSM) Patch Manager** to continuously audit simulated EC2/EKS worker nodes against PCI-DSS patch compliance baselines.
- **Phase 4: Runtime Threat Remediation via SSM Run Command (60 Mins)**
- **Runtime Threat Trigger:** Simulate a runtime compromise on a microservice (e.g., unauthorized outbound communication to a malicious IP or port scanning activity).
- **Automated Host & Network Isolation:**
 - **Amazon GuardDuty** or **VPC Flow Logs** detects anomalous egress traffic.
 - **Amazon EventBridge** catches the alert and triggers an **AWS Lambda** SOAR function that executes an **AWS Systems Manager (SSM) Run Command** document on the compromised instance to:
 - Quarantining the instance by updating its Security Group to block all inbound/outbound traffic except management ports.
 - Collect a forensic memory snapshot to an encrypted S3 bucket for analysis.

4-hour Cybersecurity Hackathon Problem Statement:

Automated DevSecOps Pipeline & Infrastructure Security Compliance for Cloud-Native Workloads

